

CYBERSECURITY INTERNSHIP REPORT

Redynox

20260

Prepared by:	Timothy Segun-Olufemi
Role:	Cybersecurity Intern
Organisation:	Redynox
Year:	2026

1. Introduction

This report documents the tasks completed during my cybersecurity internship at Redynox. The internship was designed to provide hands-on exposure to core cybersecurity disciplines including network security analysis and web application penetration testing. The objective was to move beyond theoretical knowledge and develop practical skills using real industry tools in a structured lab environment.

Two primary tasks were assigned: the first focused on network security fundamentals including traffic analysis and firewall configuration, and the second on web application vulnerability assessment and exploitation. Both tasks required the use of professional-grade tools commonly found in real security operations and penetration testing engagements.

Throughout the internship, all activities were conducted in controlled, legal environments either on personal home network infrastructure or intentionally vulnerable lab applications specifically designed for security training. The findings documented in this report reflect genuine discoveries made during the completion of each task.

2. Task Overview

2.1 Task 1: Introduction to Network Security Basics

Objective

The objective of Task 1 was to understand the fundamentals of network security by researching common network threats, implementing basic security measures on a home network, and using Wireshark to capture and analyse live network traffic. The task also required reflecting on security best practices applicable to larger, more complex network environments.

Tools and Methods Used

Firewall: Windows Defender Firewall with Advanced Security

Traffic Analyser: Wireshark 4.6.4-x64

Network Environment: Home Wi-Fi network (router + connected devices)

Method: Practical configuration, live traffic capture, and manual analysis

Steps Taken

Step 1: Researching Network Threats

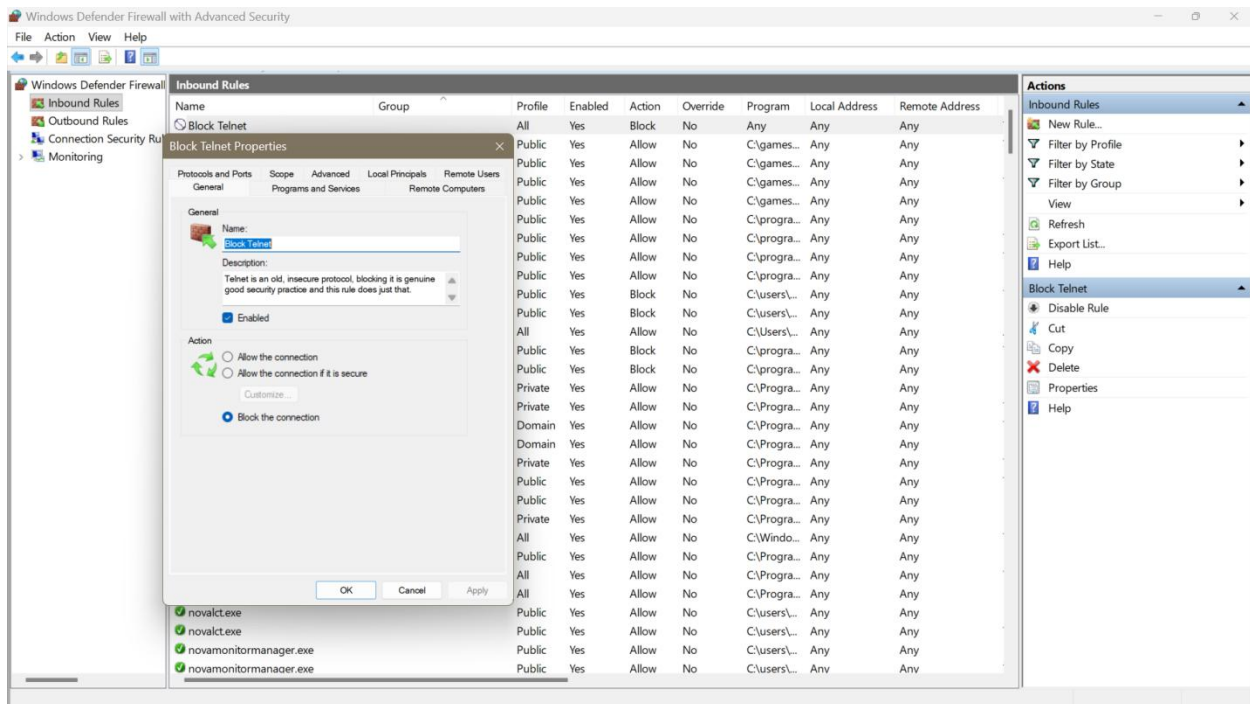
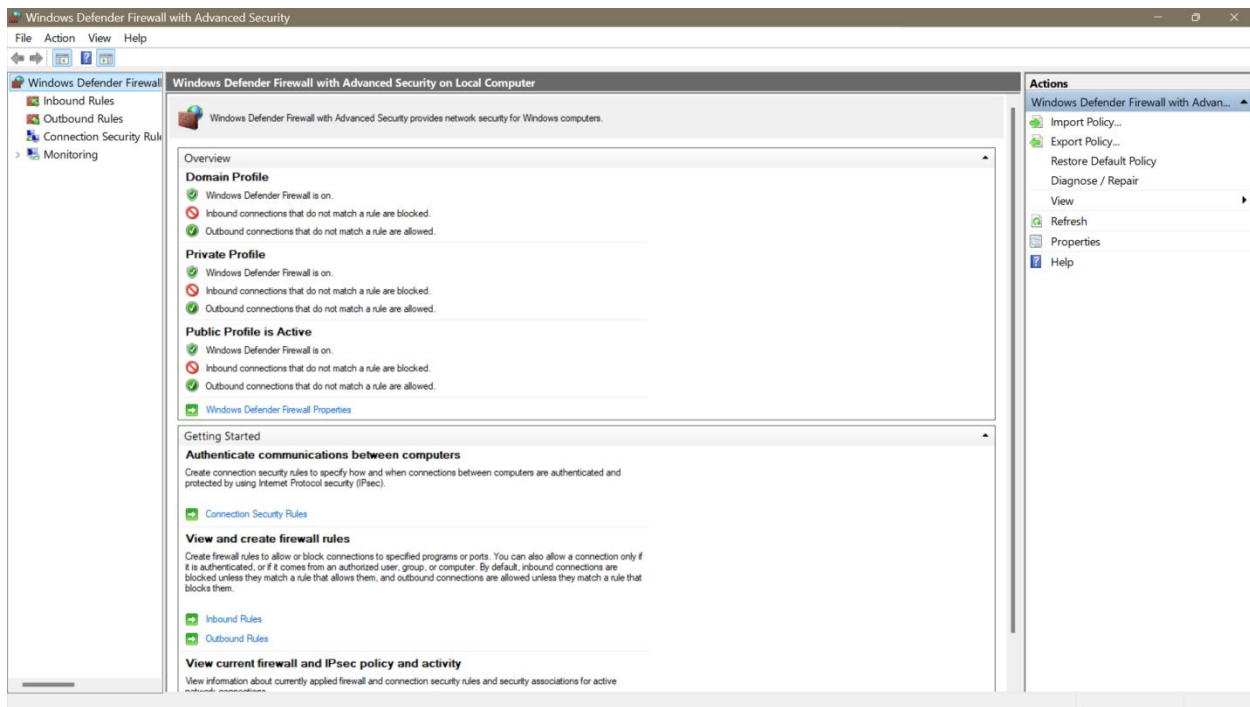
Research was conducted into the four major categories of network threats. Viruses are malicious programs that attach to legitimate files and activate when executed, spreading to other files on the system. Worms are self-replicating malware that spread across networks automatically without requiring human interaction, exploiting vulnerabilities in network services. Trojans disguise themselves as legitimate software to trick users into installation, after which they can provide attackers with remote access or deploy additional payloads. Phishing attacks use social engineering typically via email or fake websites to deceive users into revealing sensitive credentials or personal information.

Core security concepts were also studied, including how firewalls filter traffic based on defined rules, how encryption protocols such as WPA2/WPA3 protect wireless communications, and how secure network configurations reduce the attack surface of a network.

Step 2: Implementing Basic Security Measures

Windows Defender Firewall was opened using the `wf.msc` command to access the advanced security settings. The firewall was confirmed to be active across all three network profiles Domain, Private, and Public with inbound connections set to block by default. A custom inbound rule was created to explicitly block Telnet connections on TCP port 23. Telnet was selected because it is a legacy protocol that transmits all data including credentials in plain text and has no place in a secure network configuration.

The home router was also configured to use WPA2 encryption for wireless communications, the default administrator password was changed, and the default SSID was updated to avoid revealing the router manufacturer.

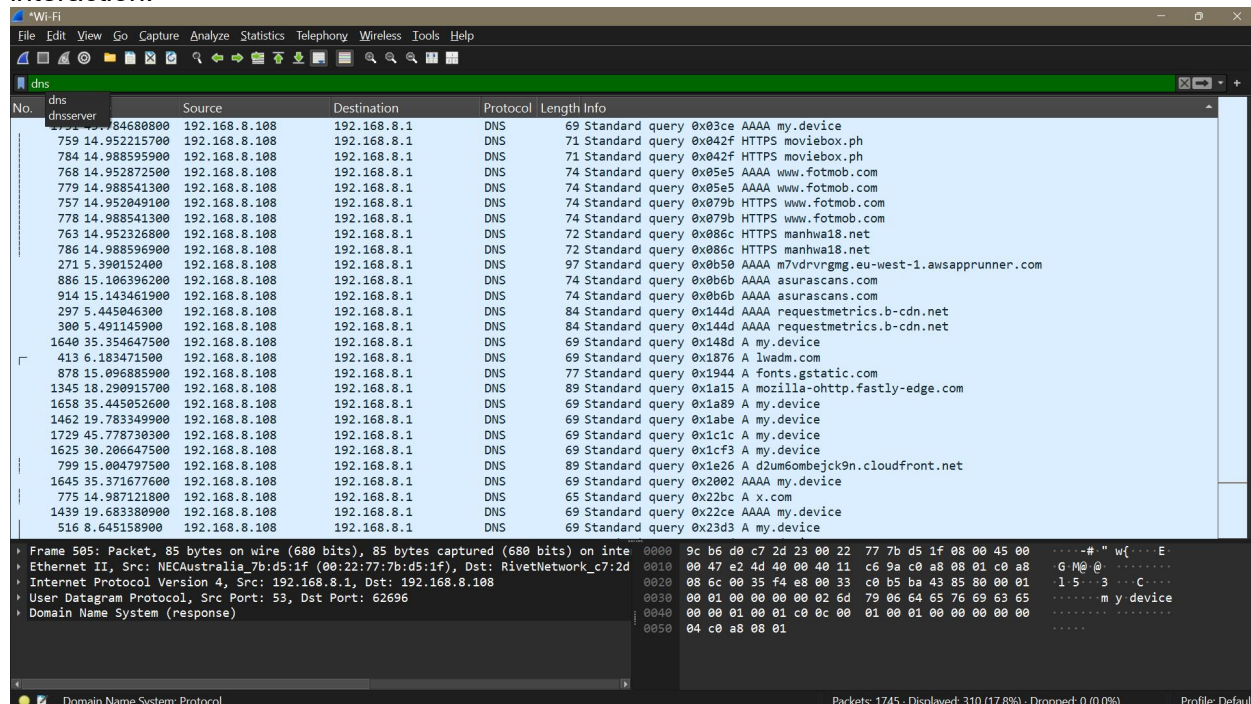


Step 3: Monitoring Network Traffic with Wireshark

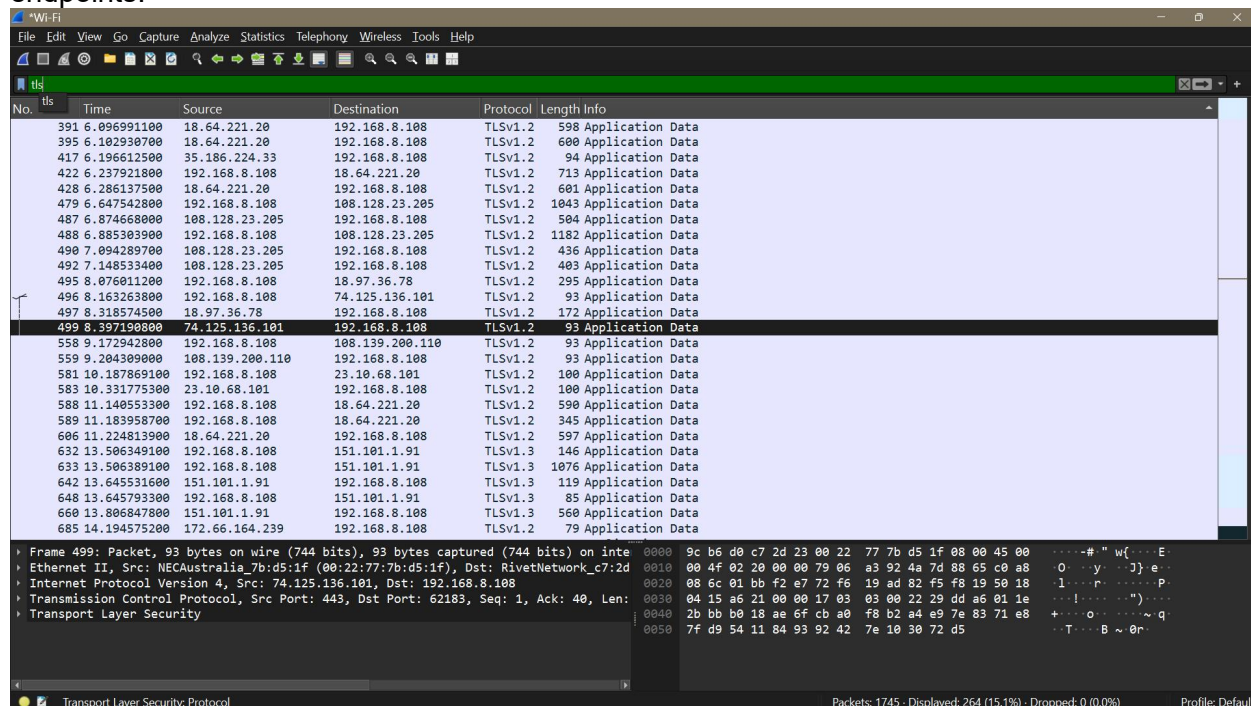
Wireshark was installed with the Npcap packet capture driver and configured to monitor the active Wi-Fi interface. A live capture session was initiated while browsing several websites to generate diverse traffic. The following filters were applied to analyse specific traffic types:

- DNS filter (dns): Revealed the computer making constant background queries to the local router at 192.168.8.1. Domains identified included fotmob.com, asurascans.com, fonts.gstatic.com, mozilla-ohhttp.fastly-edge.com, and x.com demonstrating how

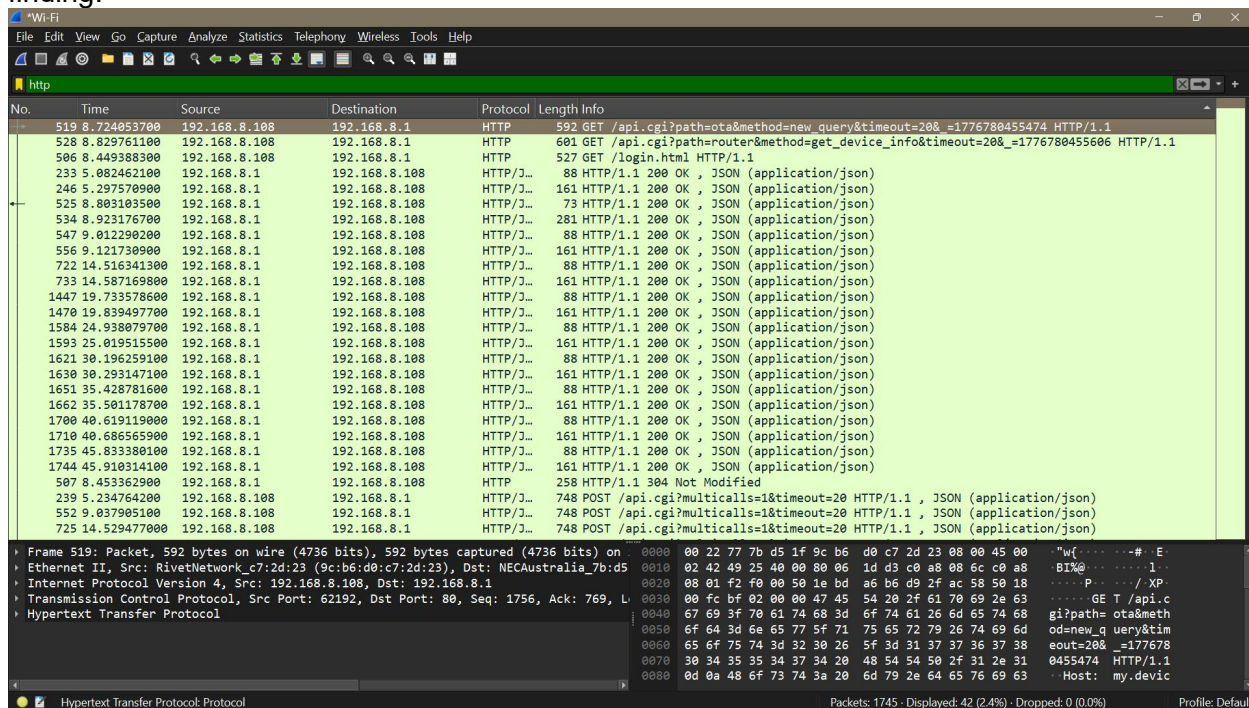
applications continuously make network requests in the background even without active user interaction.



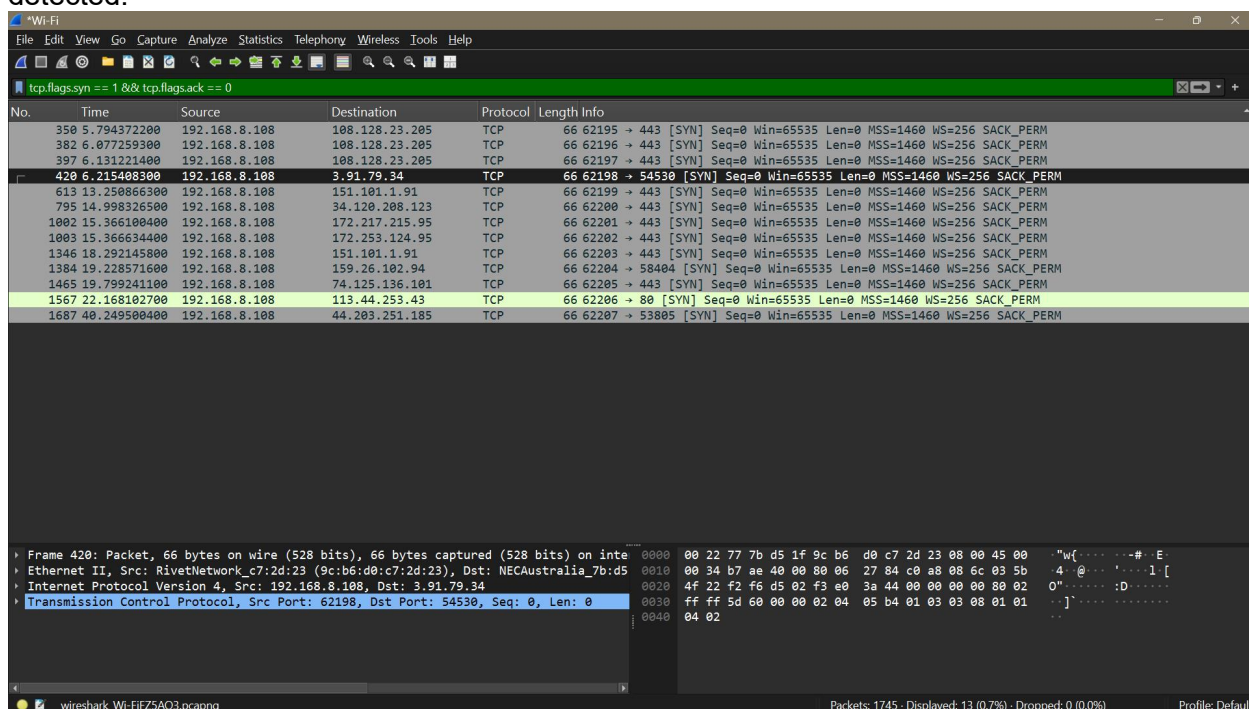
- TLS filter (tls): Showed 264 encrypted packets using TLSv1.2 and TLSv1.3 protocols. All content appeared as unreadable 'Application Data', confirming that encryption was functioning correctly for HTTPS connections to external servers including AWS and Fastly CDN endpoints.



- HTTP filter (http): Revealed unencrypted traffic between the machine and the router at 192.168.8.1, including GET requests to /login.html and API calls such as GET /api.cgi?path=router&method=get_device_info. This constituted a genuine security finding.



- SYN filter (tcp.flags.syn == 1 && tcp.flags.ack == 0): Displayed 13 outgoing TCP connection initiation packets across a 40-second window, all originating from the local machine to known cloud providers including Google, AWS, Fastly, and IBM Cloud. No inbound port scanning activity was detected.



Challenges Faced and How They Were Overcome

- Wireshark initially produced a large volume of packets that was difficult to interpret. This was resolved by applying specific display filters to isolate relevant traffic types one at a time.
- The SYN filter results initially appeared suspicious due to connections on non-standard ports. Further analysis of the destination IP addresses confirmed these were legitimate connections to known cloud services.

Results and Outcomes

The task was completed successfully with all required security measures implemented and traffic captured across all required categories. A significant real-world security finding was also identified during the Wireshark analysis:

Security Finding: Router Admin Interface Over HTTP

The router's administrative interface was found to be communicating over unencrypted HTTP (port 80) rather than HTTPS. HTTP traffic was visible in plain text in Wireshark, including API calls retrieving device information and account details. Any attacker with access to the same network could intercept these requests using Wireshark and potentially capture administrator credentials. Recommended mitigation: configure the router to enforce HTTPS for all admin interface access.

Reflection on Security Best Practices

In a larger and more complex network environment, several additional security measures would be essential. Every server both internal and external should enforce HTTPS rather than HTTP to prevent credential interception. Firewall rules should adhere to the principle of least privilege, allowing only the specific communications required between systems and blocking all others by default. Network segmentation should be implemented to isolate systems into separate zones, so that a compromise of one segment cannot freely propagate across the entire network. Automated Intrusion Detection Systems (IDS) should replace manual traffic monitoring, providing continuous analysis and alerting rather than periodic manual reviews.

Regarding security awareness, the most effective approach to educating non-technical users is through practical demonstration. Showing someone their own network activity being captured in Wireshark including readable credentials in HTTP traffic is far more impactful than theoretical explanations. Key messages to convey include: never transmit sensitive information over unsecured connections, always keep firewalls enabled, use strong and unique passwords for all network devices, and report unusual network behaviour immediately.

2.2 Task 2: Introduction to Web Application Security

Objective

The objective of Task 2 was to learn about common web application vulnerabilities by setting up an intentionally vulnerable web application, scanning it with an automated security tool, and

manually exploiting the identified vulnerabilities. The three required vulnerability classes were SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

Tools and Methods Used

Vulnerable Application: WebGoat (OWASP intentionally vulnerable web application)

Security Scanner: OWASP ZAP (Zed Attack Proxy) v2.17.0

Runtime: Java OpenJDK 25.0.2 (Eclipse Temurin)

Method: Automated scanning followed by manual exploitation

Steps Taken

Step 1: Setup

Java OpenJDK 25 was installed from Adoptium (Eclipse Temurin distribution) and verified via the command line. WebGoat was downloaded as an executable JAR file from the official OWASP GitHub repository and launched using the command `java -jar webgoat.jar`, making it accessible at `http://localhost:8080/WebGoat`. A user account was registered within the application. OWASP ZAP was installed separately and configured to run on port 8081 to avoid conflict with WebGoat's port 8080.

Step 2: Automated Vulnerability Scan

OWASP ZAP's Automated Scan feature was used to scan WebGoat at `http://127.0.0.1:8080/WebGoat`. ZAP deployed both a traditional spider and an AJAX spider to crawl the application, followed by an active scan that sent over 1,700 requests to test for vulnerabilities. The scan completed and identified the following alerts:

- SQL Injection: High severity
- Absence of Anti-CSRF Tokens (5 instances): Medium severity
- Content Security Policy (CSP) Header Not Set: Medium severity
- Missing Anti-Clickjacking Header (5 instances): Medium severity
- Spring Actuator Information Leak: Medium severity
- Cookie without SameSite Attribute: Low severity
- X-Content-Type-Options Header Missing: Low severity
- User Controllable HTML Element Attribute (XSS indicator): Low severity

Automated Scan

This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'. Please be aware that you should only attack applications that you have been specifically given permission to test.

URL to attack: Select...

Use traditional spider: ☒

Use ajax spider: ☐ If Modern ☐ with

Progress: Actively scanning (attacking) the URLs discovered by the spider(s)

Alerts

ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
623	5/21/26, 12:13:57 AM	5/21/26, 12:13:57 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		65 ms	139 bytes	4,472 bytes
624	5/21/26, 12:13:57 AM	5/21/26, 12:13:57 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		66 ms	139 bytes	4,473 bytes
625	5/21/26, 12:13:57 AM	5/21/26, 12:13:58 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		169 ms	139 bytes	4,468 bytes
626	5/21/26, 12:13:58 AM	5/21/26, 12:13:58 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		137 ms	139 bytes	4,473 bytes
627	5/21/26, 12:13:58 AM	5/21/26, 12:13:58 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		55 ms	139 bytes	4,408 bytes
628	5/21/26, 12:13:58 AM	5/21/26, 12:13:58 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		85 ms	139 bytes	4,443 bytes
629	5/21/26, 12:13:58 AM	5/21/26, 12:13:58 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		233 ms	139 bytes	4,446 bytes
630	5/21/26, 12:13:58 AM	5/21/26, 12:13:58 AM	POST	http://127.0.0.1:8080/WebGoat/register.mvc	200		323 ms	139 bytes	4,444 bytes

SQL Injection

URL: http://127.0.0.1:8080/WebGoat/register.mvc

Risk: High

Confidence: Medium

Parameter: agree

Attack: agree AND 1=1 --

Evidence:

CWE ID: 89

WASC ID: 19

Source: Active (40018 - SQL Injection)

Input Vector: Form Query

Description:

SQL injection may be possible.

Other Info:

The page results were successfully manipulated using the boolean conditions [agree AND 1=1 --] and [agree AND 1=2 --]

The parameter value being modified was stripped from the HTML output for the purposes of the comparison.

Data was returned for the original parameter.

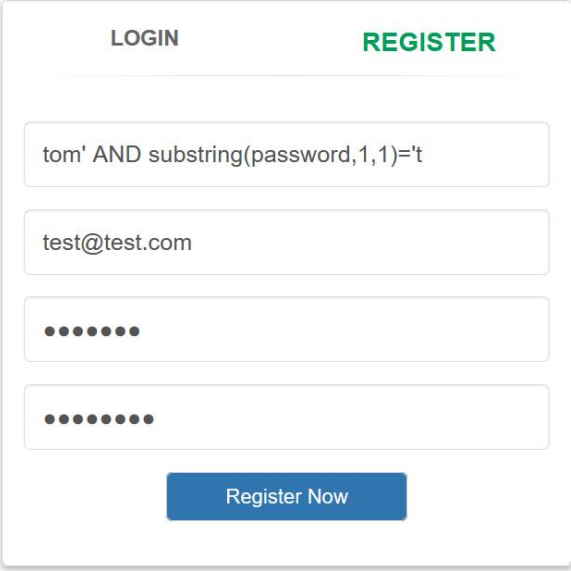
Step 3: Manual Exploitation — SQL Injection

The SQL Injection module in WebGoat was used to demonstrate manual exploitation. The exercise involved logging into an application as a user named Tom without knowing his password simulating a real-world credential theft scenario.

Initial attempts to inject directly into the login form produced generic error messages regardless of the payload, which is a common application behaviour designed to prevent information leakage. Investigation revealed that the registration form was vulnerable instead a technique known as blind SQL injection.

By injecting the payload `tom' AND substring(password,1,1)='t` into the registration username field, the application's response revealed whether the condition was true or false. When the response stated 'User already exists', it confirmed the condition was true meaning Tom's password began with the letter 't'. In a real penetration test, this process would be automated using a tool such as sqlmap to extract the complete password character by character within seconds.

Once Tom's password was determined to be `thisisasecretfortomonly` through the injection process, successful authentication was achieved demonstrating that an attacker could gain unauthorised access to any account without knowing the password.



LOGIN **REGISTER**

tom' AND substring(password,1,1)='t

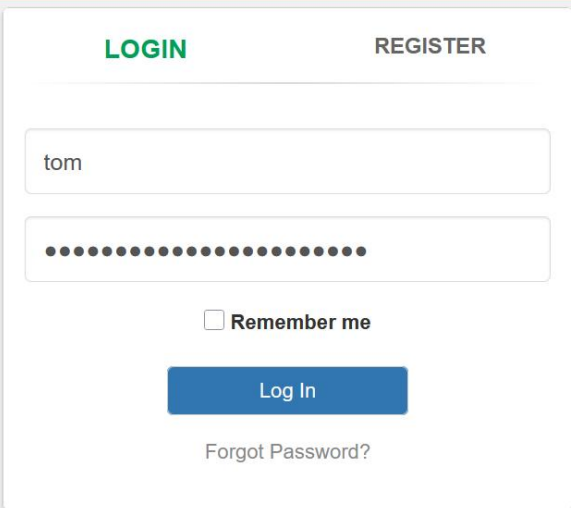
test@test.com

.....

.....

Register Now

User tom' AND substring(password,1,1)='t already exists please try to register with a different username.



LOGIN **REGISTER**

tom

.....

☐ Remember me

Log In

Forgot Password?

Congratulations. You have successfully completed the assignment.

SQL Injection — Risk and Mitigation

Risk: Allows attackers to extract, modify, or delete database contents and bypass authentication entirely.

Mitigation: Use parameterised queries or prepared statements, implement input validation, apply the principle of least privilege to database accounts, and use a Web Application Firewall (WAF).

Step 4: Manual Exploitation — Cross-Site Scripting (Stored XSS)

The Cross-Site Scripting (Stored) module in WebGoat was used to demonstrate XSS exploitation. Stored XSS is the most dangerous form of XSS because the malicious script is saved to the server's database and executes automatically in every visitor's browser when the affected page loads.

The following JavaScript payload was injected into the application's comment field:

```
<script>webgoat.customjs.phoneHome()</script>
```

Rather than storing this as plain text, the application executed the script. The phoneHome() function made a request to WebGoat's server and returned a unique response value confirmed via the browser's developer tools network panel showing a POST request to phone-home-xss with the response 'phoneHome Response is 994930725' and lessonCompleted: true.

In a real attack scenario, this payload could be replaced with a cookie-stealing script that redirects victims to an attacker-controlled server, effectively hijacking authenticated sessions without the victim's knowledge.



John Doe uploaded a photo.

24 days ago



```
<script>webgoat.customjs.phoneHome()</script>
```



intern 2026-05-24, 06:23:27



intern 2026-05-24, 06:21:26



intern 2026-05-24, 06:20:09



guest 2026-05-24, 05:46:28

Can you post a comment, calling webgoat.customjs.phoneHome() ?



secUrity 2026-05-24, 05:46:28
Comment for Unit Testing

Watching in your browser's developer tools or your proxy, the output should include a value starting with 'phoneHome Response is'
Put that value below to complete this exercise. Note that each subsequent call to the *phoneHome* method will change that value. You may need to ensure you have the most recent one.

994930725

Submit

Yes, that is the correct value (note, it will be a different value each time the phoneHome endpoint is called).

Inspector

Console

Debugger

Network

Style Editor

Performance

Memory

Storage

Accessibility

Application

Filter URLs

|| + 🔍 🔄

All

HTML

CSS

JS

XHR

Fonts

Images

Media

WS

Other

☐ Disable Cache

No Throttling

⚙

Status	Meth...	Domain	File	Initiator	Type	Transferred	Size	Headers	Cookies	Request	Response	Timings	Stack Trace
200	GET	127.0.0.1:8...	stored-xss	jquery-2.1.4.m...	json	1.05 kB	891 B	Filter properties					
200	GET	127.0.0.1:8...	CrossSiteScriptingStored.lesson	jquery.min.js2...	json	602 B	396 B	JSON					
200	GET	127.0.0.1:8...	hint.mvc	jquery.min.js2...	json	25.28 kB	25.12...	lessonCompleted: true					
200	POST	127.0.0.1:8...	phone-home-xss	jquery.min.js2...	json	447 B	285 B	feedback: "Congratulations. You have successfully completed the assignment."					
500	POST	127.0.0.1:8...	phone-home-xss	jquery.min.js2...	json	21.51 kB	21.37...	feedbackArgs: null					
200	POST	127.0.0.1:8...	phone-home-xss	jquery.min.js2...	json	448 B	286 B	output: "phoneHome Response is 994930725"					
200	GET	127.0.0.1:8...	lessonmenu.mvc	jquery.min.js2...	json	8.96 kB	8.80 ...	outputArgs: null					
200	GET	127.0.0.1:8...	CrossSiteScriptingStored.lesson	jquery.min.js2...	json	602 B	396 B	assignment: "DOMCrossSiteScripting"					
200	GET	127.0.0.1:8...	lessonmenu.mvc	jquery.min.js2...	json	8.96 kB	8.80 ...	attemptWasMade: true					
105 requests 563.96 kB / 343.94 kB transferred Finish: 1.69 min DOMContentLoaded: 233 ms load: 261 ms													

XSS — Risk and Mitigation

Risk: Enables session hijacking, credential theft, website defacement, and redirection to phishing pages affecting all users who visit the compromised page.

Mitigation: Implement output encoding for all user-supplied data, enforce a strict Content

Security Policy (CSP) header, validate and sanitise all inputs on both client and server side.

Step 5: CSRF — Identified via Automated Scan

Cross-Site Request Forgery (CSRF) was identified by OWASP ZAP during the automated scan, which flagged 5 instances of 'Absence of Anti-CSRF Tokens' at medium severity. The scan evidence showed HTML forms including the WebGoat login form submitting POST requests without any CSRF token to verify the request originated from a legitimate user action.

CSRF attacks work by tricking an authenticated user's browser into sending a forged request to a trusted application. Because the browser automatically includes session cookies with every request, the server cannot distinguish between a legitimate user action and a forged one. For example, an attacker could embed a hidden form on a malicious website that automatically submits a fund transfer request to a banking application the victim is logged into.

CSRF — Risk and Mitigation

Risk: Allows attackers to perform unauthorised actions on behalf of authenticated users including changing passwords, transferring funds, or modifying account settings.

Mitigation: Implement CSRF tokens on all state-changing forms, use the SameSite cookie attribute, verify the Origin and Referer headers on the server side, and require re-authentication for sensitive operations.

Challenges Faced and How They Were Overcome

- WebGoat initially failed to launch due to a port conflict on 8080. This was resolved by ensuring no other services were occupying that port before starting WebGoat.
- OWASP ZAP could not use port 8080 because WebGoat was already using it. ZAP was reconfigured to use port 8081, and both tools ran simultaneously without issue.
- The SQL Injection exercise did not respond to standard login bypass payloads. Through analysis of the application behaviour, it was determined that the vulnerability existed in the registration form rather than the login form, requiring a blind injection approach.
- The dedicated CSRF exercise module was not available in the installed version of WebGoat. This was resolved by using the OWASP ZAP automated scan findings as documented evidence of CSRF vulnerability, which is a valid and widely accepted approach in professional penetration testing reports.

Results and Outcomes

All three required vulnerability classes were successfully identified and demonstrated. SQL Injection and Stored XSS were manually exploited with confirmed results. CSRF was identified and documented through automated scanning. All findings were supported by screenshots taken throughout the process. The task demonstrated a complete web application security assessment workflow from setup and automated scanning through to manual exploitation and documentation.

3. Conclusion

This internship provided a highly practical introduction to two of the most fundamental disciplines in cybersecurity: network security and web application security. Working through both tasks using real industry tools gave me direct experience with the same workflows and methodologies used by professional security analysts and penetration testers.

From Task 1, I developed a working understanding of how network traffic behaves in practice not just in theory. Capturing live traffic with Wireshark and identifying an actual security vulnerability in my own router's administrative interface was a particularly valuable experience. It demonstrated that security weaknesses are not hypothetical; they exist in everyday environments and can be discovered with the right tools and knowledge.

From Task 2, I gained hands-on experience with the full lifecycle of a web application security assessment setting up a target environment, running automated scans, interpreting the results, and manually verifying and exploiting the findings. Working through SQL Injection and XSS exploitation in WebGoat gave me a concrete understanding of how these attacks work at a technical level, which is essential for both offensive and defensive security roles.

Professionally, this internship strengthened my ability to work methodically through technical problems, document findings clearly, and think from an attacker's perspective a mindset that is central to effective cybersecurity work. The experience has reinforced my commitment to pursuing penetration testing and offensive security as a career path, and has given me a foundation of practical skills to build on.

4. Closing Remarks

I would like to acknowledge the Redynox team for structuring the internship tasks in a way that balanced foundational learning with genuine practical application. The task design requiring not just research but actual implementation, tool usage, and documentation made this a meaningful and educational experience.

For future interns approaching these tasks, I would recommend setting up the lab environment early and ensuring all tools are installed and verified before beginning the technical exercises. Many of the challenges encountered during this internship related to environment setup rather than the security concepts themselves, and addressing these early avoids frustration later.

I would also suggest that future iterations of the internship include a dedicated CSRF exploitation exercise, as the practical demonstration of CSRF attacks reinforces understanding in a way that scanner-based identification alone does not fully achieve.

Overall, this internship was a valuable step in my development as a cybersecurity professional, and I am grateful for the opportunity to apply my knowledge in a structured and supportive environment.